PRODUCT & TECHNICAL GUIDE

Slidecraft

Project documentation

Setup, architecture, features, security, and API reference

React · Express · MongoDB · Gemini (optional)

About this guide

A practical guide to installing, configuring, running, and extending the current Slidecraft application. It documents implemented behavior and calls out features that are not yet available.

Inside

- Product overview · Technology and architecture · Features and limitations
- Installation · Environment configuration · Local and production operation
- User workflow · Authentication and security · API reference
- PDF export · Project structure · Troubleshooting

Slidecraft

Project documentation

Slidecraft is a presentation-building web application. Users register or sign in, describe a presentation, choose an audience and visual style, set the slide count and talk duration, then preview the generated slides and download them as a PDF.

This guide covers the current implementation in this repository. It does not describe features that are planned but not implemented.

Contents

1. Product overview
2. Technology and architecture
3. Features and current limitations
4. Requirements and installation
5. Environment configuration
6. Running locally and production build
7. User workflow
8. Authentication and security
9. Slide generation API
10. PDF export
11. Project structure
12. Troubleshooting

1. Product overview

Slidecraft has two connected applications:

- A React single-page application for authentication, deck setup, slide preview, navigation, speaker notes, and PDF downloads.
- An Express API for readiness checks, account registration and login, session lookup and logout, and authenticated slide generation.

MongoDB Atlas stores user account records. When a Gemini API key is configured, the server can request AI-generated slide content. Without the key, a built-in local outline generator remains available.

Generated presentations currently live in the browser's application state. They are not saved as presentation documents in MongoDB and will not automatically reappear after a refresh.

2. Technology and architecture

Stack

- React 18 and Vite for the browser application.
- Node.js and Express 4 for the API and production static-file serving.
- MongoDB Atlas and Mongoose for accounts.
- bcryptjs for password hashing.
- JSON Web Tokens in HTTP-only cookies for sessions.
- Google Gemini 2.5 Flash, optional, for AI slide content.
- jsPDF for client-side PDF generation.
- Helmet, express-rate-limit, and validator for security headers, request throttling, and validation.

Request flow

1. Vite serves the React application on http://localhost:5173 during development.
2. Vite proxies requests beginning with /api to Express on http://localhost:5000.
3. Express checks MongoDB and JWT configuration and returns these readiness flags from /api/health.
4. The client checks /api/auth/me before showing the private workspace.
5. Registration or sign-in sets the slidecraft_session cookie.
6. The browser sends that cookie to protected API routes automatically.
7. Slide content is either generated locally or requested from Gemini on the server.
8. jsPDF formats the current deck in the browser and downloads a PDF file.

3. Features and current limitations

Implemented

- Login-first access with separate /login, /register, and /signed-out screens.
- Email/password registration, sign-in, session checking, and sign-out.
- Presentation topic, audience, style, slide count, and duration inputs.
- Local outline generation without an AI key.
- Optional AI slide content through Gemini.
- Slide navigation, animated transitions, and speaker notes.
- Landscape 16:9 PDF download with one slide per page.
- Responsive layouts and reduced-motion support.

Not implemented

- Saving or listing presentations across browser sessions.
- Editing individual generated slide content.
- PowerPoint export, collaboration, or presentation mode.
- Automatic research, source citations, or fact verification.
- Password reset, email verification, or multi-factor authentication.

Review AI-generated content and verify factual claims before presenting it. The local generator is an outline starter, not a research or fact-checking system.

Requirements

- Node.js 18 or newer.
- npm.
- A MongoDB Atlas cluster and database user to enable accounts.
- A Google AI Studio API key only if AI-generated slide content is desired.

Install

Open PowerShell in the project directory and run:

```
npm install
```

The dependency lockfile is package-lock.json; keep it when installing to reproduce the project dependency versions.

5. Environment configuration

Copy the example environment file:

```
Copy-Item .env.example .env
```

Edit .env locally. Never paste its contents into chat, check it into source control, or expose it in browser code.

Variables

Variable	Required	Description
MONGODB_URI	For accounts	MongoDB Atlas connection string. Use a database user, select a database such as slidecraft, and URL-encode special characters in the password.
JWT_SECRET	For accounts	A private random secret of at least 32 characters.
GEMINI_API_KEY	No	Optional Gemini API key. The key stays on the server and is sent to Google's Generative Language API from the backend.
PORT	No	Express port. Defaults to 5000.
NODE_ENV	No	Set to production when serving the production build.

Example structure only—replace every placeholder locally:

```
GEMINI_API_KEY=
```

```
MONGODB_URI=mongodb+srv://<database-user>:<url-encoded-password>@<cluster-host>/slidecraft?
retryWrites=true&w=majority
```

```
JWT_SECRET=<random-secret-at-least-32-characters>
```

```
PORT=5000
```

Generate a random session secret with Node.js:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

Create a Gemini API key in Google AI Studio. Free-tier access, limits, and availability are controlled by Google and can vary by region and account.

After changing .env, restart the development server for the new values to load.

6. Running locally and production build

Development

```
npm run dev
```

Open:

- Website: http://localhost:5173/login
- API health: http://localhost:5000/api/health

npm run dev starts both the API and Vite development server. Keep the terminal process running while using the app.

Production build

Build the React client:

```
npm run build
```

Serve the built client through Express:

```
$env:NODE_ENV = "production"
```

```
npm start
```

The Express server serves client/dist in production. Deploy the application behind HTTPS; production session cookies are marked Secure.

7. User workflow

1. A visitor opens the site and is directed to sign in.
2. A new visitor selects Create an account and registers with a name, email, and password.
3. An existing user signs in with their email and password.

4. On successful authentication, the slide workspace appears.
5. Enter a topic and choose an audience, visual style, slide count, and talk duration.
6. Select Generate my slides.
7. Review the slide preview, navigate between slides, and show or hide speaker notes.
8. Select Export PDF to download a PDF with one slide per page.
9. Select Sign out to clear the session and return to the signed-out page.

The browser checks the session at page load, when the tab regains focus, and every five minutes. The server independently validates authentication before generating slides.

8. Authentication and security

Registration validation

- Name: 1–80 characters after normalization and trimming. Letters, marks, numbers, spaces, apostrophes, periods, and hyphens are accepted.
- Email: validated and normalized to lowercase; up to 254 characters.
- Password: at least 12 characters, maximum 72 UTF-8 bytes to fit bcrypt's input limit.
- Passwords are hashed using bcrypt with 12 salt rounds; plaintext passwords are not stored.
- Email is unique in MongoDB.

Sessions

- Session cookie name: slidecraft_session.
- JWT algorithm: HS256, with a fixed issuer and audience.
- Expiration: seven days.
- Cookie flags: HttpOnly, SameSite=Strict, path /, and Secure in production.
- Sign-out clears the cookie in the browser.

Other controls

- Registration: 5 requests per IP per hour.
- Sign-in: 10 requests per IP per 15 minutes.
- Request JSON size: limited to 16 KB.
- Helmet sets security headers and a Content Security Policy.
- /api/generate requires a valid session; hiding the workspace in React is not the only access-control layer.
- API keys belong in the server-side .env, not React source or client-side environment variables.
- Use a narrow Atlas IP access list in normal operation. 0.0.0.0/0 permits access attempts from any IP and should not be left enabled unnecessarily.
- Rotate any secret that was shared, committed, or otherwise exposed.

Rate limits use the default in-memory store. For a multi-instance production deployment, use a shared rate-limit store and review proxy/IP configuration.

9. Slide generation API

All responses are JSON. Error responses use an error property. The client and API are served from the same origin in the standard setup.

`GET /api/health`

Returns operational and configuration flags without returning credentials:

```
{
  "status": "ok",
  "aiEnabled": false,
  "authEnabled": false,
  "mongoConnected": false,
  "mongoUriConfigured": false,
  "jwtConfigured": false
}
```

authEnabled is true only when MongoDB is connected and the JWT secret is valid. aiEnabled reflects whether a Gemini key is configured.

`POST /api/auth/register`

Creates an account and sets the session cookie.

Request:

```
{
  "name": "Jane Smith",
  "email": "jane@example.com",
  "password": "a-long-passphrase"
}
```

Responses: 201 created, 400 invalid fields, 409 duplicate email, 429 rate limit, 503 account service not configured, or 500 unexpected server error.

`POST /api/auth/login`

Verifies an existing account and sets the session cookie.

Request:

```
{
  "email": "jane@example.com",
  "password": "a-long-passphrase"
}
```

Responses: 200 signed in, 400 invalid request, 401 incorrect credentials, 429 rate limit, 503 account service not configured, or 500 unexpected server error.

`GET /api/auth/me`

Returns the signed-in user as { "user": { "id": "...", "name": "...", "email": "..." } }, or { "user": null } when no valid session is present. It never returns a password hash.

`POST /api/auth/logout`

Clears the session cookie and returns { "ok": true }.

`POST /api/generate`

Requires a valid session cookie.

Request:

```
{
  "topic": "The future of renewable energy",
  "audience": "Students",
  "slideCount": 8,
  "style": "Modern & minimal",
  "duration": 10
}
```

Validation: topic is 2–160 characters; audience is 2–100 characters; slide count is an integer from 3–20; style is a string up to 50 characters; duration is an integer from 1–90 minutes.

The response contains a title, an array of slides, and a mode of ai or local. Each slide has a title, subtitle, bullet list, and speaker notes. Unauthenticated requests return 401; invalid fields return 400; an AI service failure returns 502.

With GEMINI_API_KEY, Express calls Gemini 2.5 Flash from the server. Without the key, Express uses its built-in local outline templates. The local mode creates an outline rather than topic-researched content.

10. PDF export

The Export PDF action uses jsPDF in the browser to create and download a PDF without an additional server request.

- Each presentation slide becomes one PDF page.
- Page format is landscape 16:9 (960 × 540 points).
- Colors are selected from the visual style.
- Text is wrapped to fit the slide layout; lengthy titles and bullet lists use smaller type where needed.
- The export does not include speaker notes, editing controls, or slide-transition animations.
- The downloaded filename is derived from the presentation title.

If export fails, the page displays an error and the button returns to its ready state.

11. Project structure

```
.
%%%   client/
%     %%%   index.html
%     %%%   vite.config.js
%     %%%   src/
%           %%%   App.jsx                 #  Authentication,  works
%           %%%   main.jsx                #  React  entry  point
%           %%%   styles.css              #  UI,  slide  themes,  ani
%%%   server/
%     %%%   index.js                      #  Express  API,  MongoDB
%%%   docs/
%     %%%   PROJECT-DOCUMENTATION.md
%%%   .env.example                        #  Safe  configuration  te
%%%   .gitignore                          #  Excludes  .env,  depend
%%%   package.json
%%%   package-lock.json
```

12. Troubleshooting

Account form says MongoDB is not configured

1. Check that .env is in the project root beside package.json.
2. Set MONGODB_URI to the connection string copied from Atlas. Do not retain placeholder angle brackets.
3. URL-encode special characters in the database password.
4. Confirm the database username/password are for a MongoDB database user, not an Atlas website login.
5. In Atlas Network Access, allow the development machine's current public IP.
6. Set JWT_SECRET to a random value of at least 32 characters.
7. Restart npm run dev, then reload /login.
8. Check the API at http://localhost:5000/api/health. It reports whether the URI is configured, MongoDB connected, and the JWT secret is configured without revealing secret values.

Do not use 0.0.0.0/0 as a permanent production allowlist.

Gemini is not enabled

Set GEMINI_API_KEY in the root .env file and restart the server. The health endpoint reports aiEnabled. If no key is configured, generation remains available in local outline mode after sign-in.

API or page does not load

- Ensure npm run dev is still running.
- Check ports 5173 (Vite) and 5000 (Express).
- Confirm Vite can proxy /api to the Express server.
- Read the terminal output for MongoDB connection failures or server startup errors.

PDF does not download

- Generate a presentation first; export is only available when a deck exists.
- Allow downloads for the local site in the browser.
- Retry after the export button leaves its “Preparing PDF…” state.
- Check the browser console for JavaScript errors.

Useful checks

```
npm run build
```

```
npm audit
```

```
Invoke-RestMethod http://localhost:5000/api/health
```

Document maintenance

Update this guide whenever routes, environment variables, security behavior, or user-facing functionality changes. The PDF version is generated from this Markdown source using npm run docs:pdf.